

Attested DNS

Presentation to the CCC-Attestation group

Amaury Chamayou (Microsoft, Azure Research: Security and Privacy)

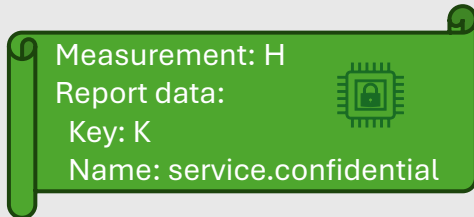
Confidential Services Today

Services are upgrading to Confidential Computing to leverage hardware-enforced isolation provided by modern server-class CPUs

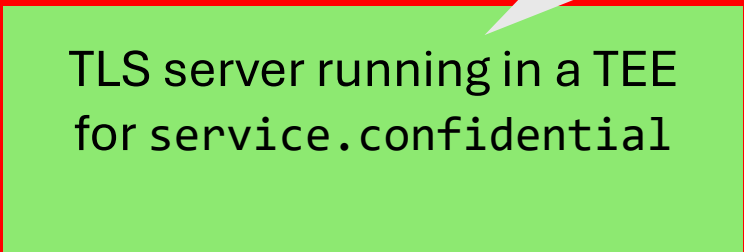
- Services run in **trusted execution environments** (TEE): protects code & data from the rest of the cloud
- The hardware signs **remote attestations**: relying parties can authenticate and authorize services running in TEEs

At TEE launch,

- *Generate fresh keypair (K , S)*
- *Generate attestation report binding the hardware platform, service identity, and code, and its public key K*

A green rectangular box with rounded corners and a scroll-like top-left corner, containing text and a chip icon.

Measurement: H
Report data:
Key: K
Name: service.confidential

A light green rectangular box with a speech bubble tail pointing to the attestation report box above it.

TLS server running in a TEE
for service.confidential

Untrusted Host

Secure Communication with Confidential Services

Goal: **All connections with/between confidential services should end within TEEs that meet their service definition (right code, right platform).**

Attestation After TLS Handshake



Benefits

- Unmodified transport stack

Problems

- **ad hoc client**
- Extra round trip

Attestation During TLS Handshake

Benefits

- Can rely on transport security

Problems

- Either modify TLS handshake, or embed attestation in existing credentials (certificates, tokens)
- Requires **ad hoc client & server**

Attestation Before TLS



Attested DNS

Attested DNS in one slide

- Services identified by **domain names**
- Decide **which TEEs** to trust for a name
 - Service **definitions** evaluated against **attestation reports**
- Authoritative DNS server **in TEE**
 - securely **enforce definitions** when a TEE joins a service
- DNS standards and infrastructure (resolvers)
 - **distribute** and **cache** TEEs **TLS keys**
- A transparent, auditable log of the confidential service fabric

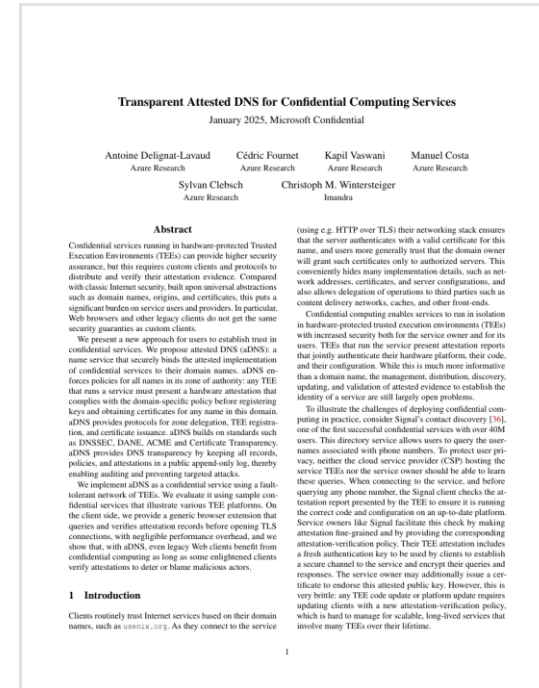
Attested DNS Design

Transparent Attested DNS for Confidential Computing Services at Usenix Security'25

- Protocols for registration, zone delegation, certificate issuance based on Internet Standards (DNS, DNSSEC, ACME, DANE, TLS)
- Increased security for all clients
 1. Enlightened clients can fetch and verify attestations over DNS
 2. DNSSEC clients get their attested keys in signed records
 3. PKIX clients get their attested keys in certificates issued after DNS name validation
 4. Anyone can audit the attested DNS log

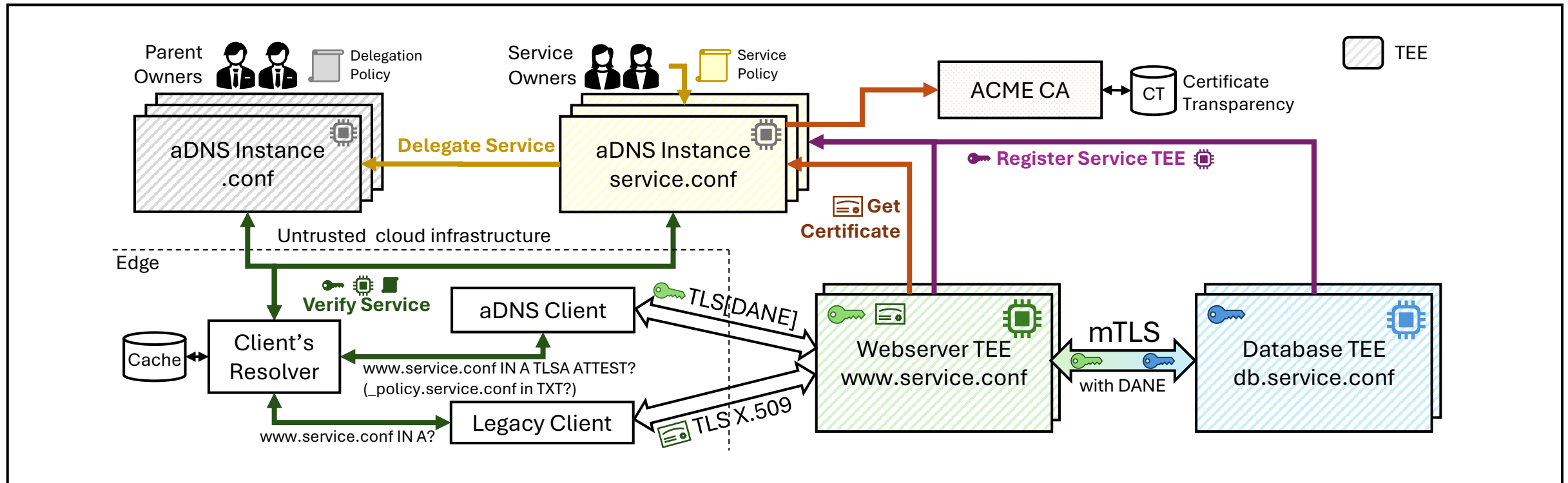
Logical progression:

- Why should I trust your platform? ➔ Confidential Containers
- Why should I trust your code/policy? ➔ CTS/Signing Transparency
- Why should I trust your service? ➔ Attested DNS



1st Implementation (Research Prototype, SGX)

1. Authoritative DNS server on top of CCF for replication & security
2. Web browser extension that verifies attestations in parallel with TLS session, with **no latency overhead**.
3. We registered attestations and issued records & certificates for various confidential platforms and services.



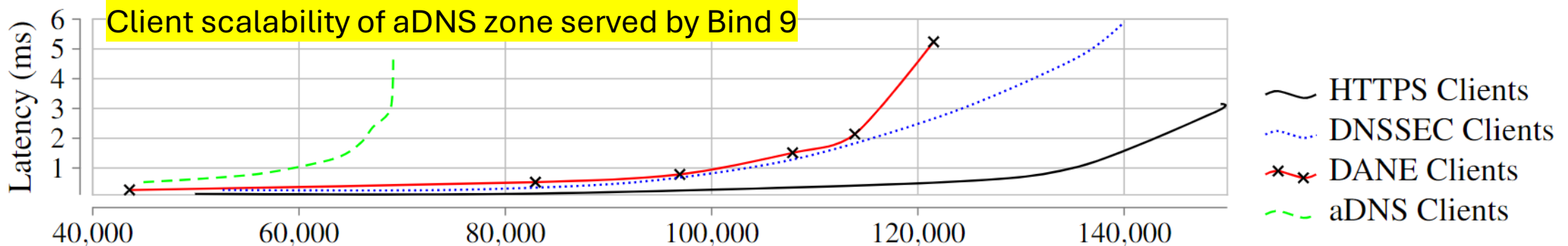
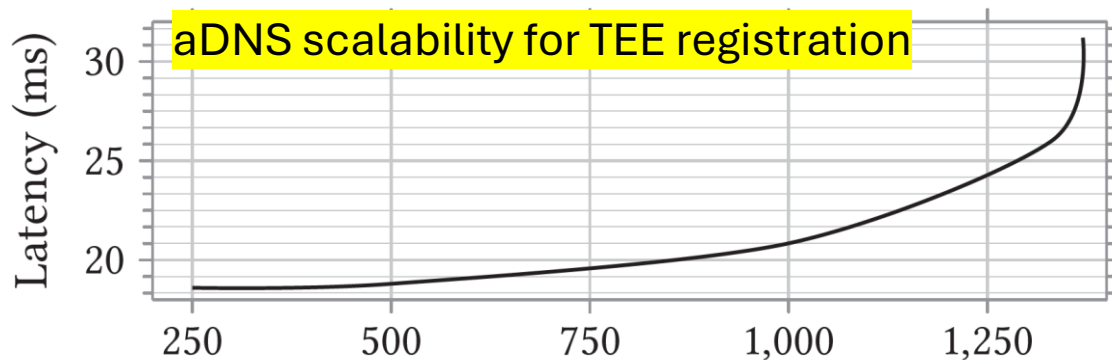
1st Implementation—Client Evaluation

	HTTPS	DNSSEC	RA-TLS	aDNS
DNS Resolution	1ms	2ms	1ms	3ms
TLS Handshake	40ms	40ms	110ms	38ms
- Attestation (SGX)	-	-	-	27ms
- Attestation (SEV)	-	-	-	13ms
- Attestation (RAS)	-	-	71ms	-
Time to first byte	41ms	42ms	111ms	41ms

1st Implementation—Server Evaluation

Endpoint	Context	Latency
/register-service	Register complete Intel SGX report	263ms
/register-service	Register SGX, missing collaterals	2.1s
/register-service	Register AMD SEV-SNP TEE	218ms
/get-certificate	Get Let's Encrypt certificate	72ms (+7.5s)
/resign	Re-sign zone with 454 records	180ms

C++ Class	Description	LoC
Ravl	Remote attestation validation library	7117
RFC1035	Base DNS record types	1278
RFC4034	DNSSEC record types	1353
RFC*	NSEC3, ETLS, DANE, CAA	673
Resolver	DNSSEC implementation	2472
CCFDNS	Endpoints (query, register...)	3248
*	Serialization and utilities	1146
CCF	Confidential Consortium Framework	~64K
	CCF Dependencies (JSON, HTTP...)	~125K
	Total Enclave TCB	~200K

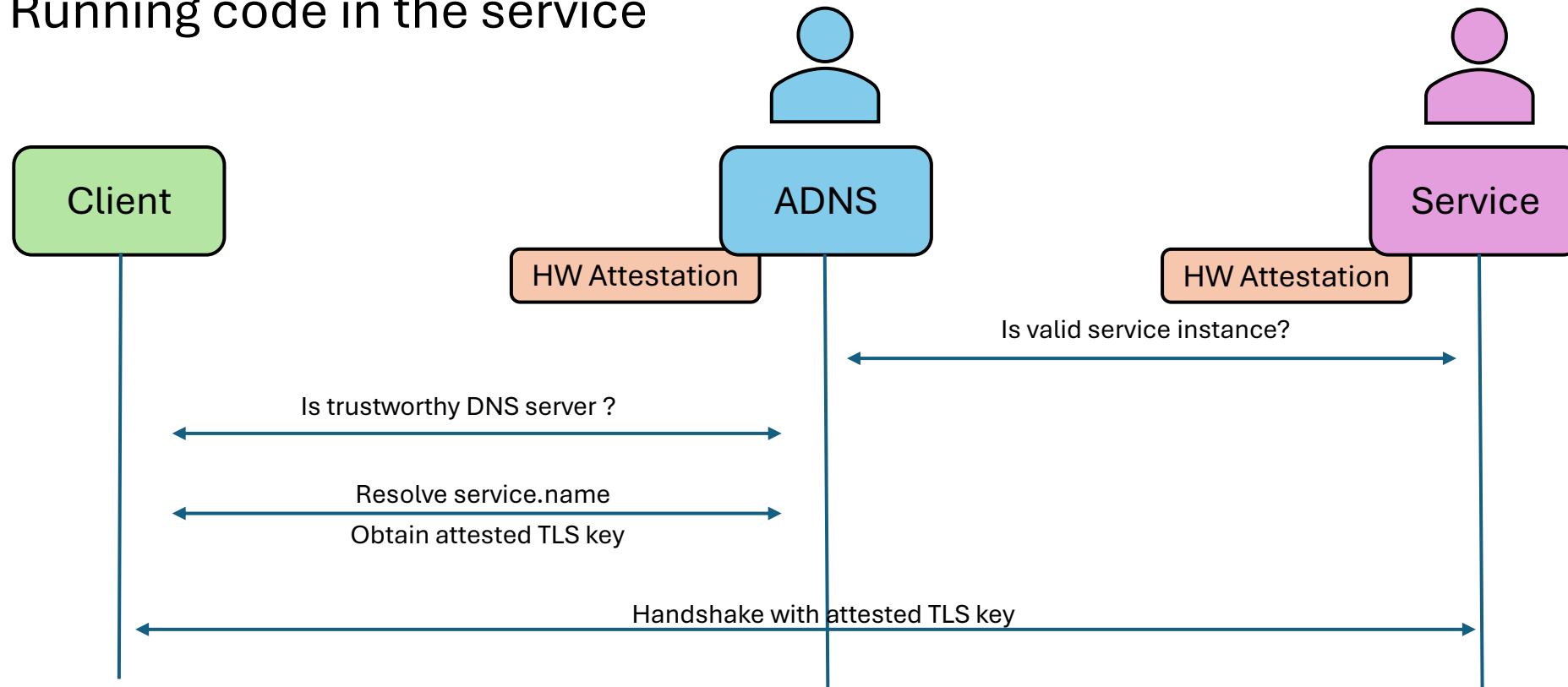


2nd Implementation

- Proof of Concept using Confidential Containers
 - **Confidential Azure Container Instances (AMD SEV-SNP)**
- Formalise transparent service and platform definitions
 - Rego policy language
- Support standard clients using OpenSSL
 - Patched OpenSSL to handle TLSA resolution and DANE transparently
- Demonstrate low footprint, client-side trust bootstrap
 - KSK from Attestation + Policy

Service **Discovery** and **Trust**

- Discovery: DNS protocol (already solved!)
- Trust
 - Platform running the service
 - Running code in the service

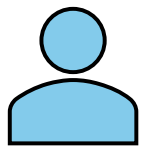


Proof of Concept - Steps

- As an operator, set up an ADNS server
- As an operator, set a Platform Definition
- As a service owner, set a Service Definition
- As a service instance, register with ADNS
 - **IP** and **TLS key**
- As a client
 - Bootstrap trust in ADNS
 - Discover the service IP and TLS key
 - Establish TLS session to service using resolved key
- No code change to the client application required!

Setup an ADNS server – part I: startup

- ADNS is a CCF application
 - Constitution: rules governing changes
 - Member identities
- Technical Configuration
 - Address, zone, etc
- Run in one or more Confidential Containers
 - Ideally $2f+1$ containers to tolerate f failures



As the ADNS operator

Setup an ADNS server – part II: authorization

- Using CCF governance
 - One-time setting of Platform Definition Authorization Policy
 - One-time setting of Service Definition Authorization Policy
 - Per-service (test.e2e.acidns10.attested.name)

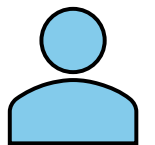
```
package policy

default allow := false

allowed_issuer if {
    input.phdr.cwt.iss == "did:x509:0:sha256:GVODzyvFbgv9XBTVsEWmwv25g4pSpabzo-EC_QEQThc::eku:1.3.6.1.5.5.7.3.36"
}

allowed_subject if {
    input.phdr.cwt.sub == "PlatformPolicy"
}

allow if {
    allowed_issuer
    allowed_subject
}
```



As the ADNS operator

Set a Platform Definition

- Authenticated request to set Platform Definition
- Gated by Platform Definition Authorization Policy
- Baseline for all services that want to register

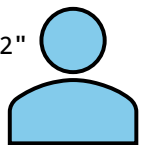
```
package policy
default allow := false

product_name_valid if {
    input.attestation.product_name in ["Milan", "Genoa"]
}
reported_tcb_valid if {
    input.attestation.reported_tcb.hexstring in ["db18000000000004", "541700000000000a"]
}
amd_tcb_valid if {
    product_name_valid
    reported_tcb_valid
}

uvm_did_valid if {
    input.attestation.uvm_endorsements.did == "did:x509:0:sha256:I__iuL25oXEVFdTP_aBLx_eT1RPHbCQ_ECBQfYZpt9s::eku:1.3.6.1.4.1.311.76.59.1.2"
}
uvm_feed_valid if {
    input.attestation.uvm_endorsements.feed == "ContainerPlat-AMD-UVM"
}
```

```
uvm_svn_valid if {
    input.attestation.uvm_endorsements.svn >= "101"
}
uvm_valid if {
    uvm_did_valid
    uvm_feed_valid
    uvm_svn_valid
}

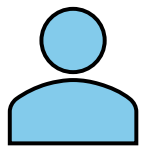
allow if {
    amd_tcb_valid
    uvm_valid
}
```



As the ADNS operator

Demo: Set up an ADNS server and Platform Definition

```
Attested DNS (10.0.1.4) x + v - □ x  
root@SandboxHost-638950921939441370 [ ~/workspace/ccfdns/demo/adns ]# cat PlatformDefinitionAuthz.rego |
```



As the ADNS operator

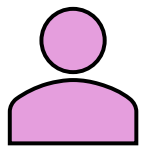
Set a Service Definition for `test.e2e.acidns10.attested.name`

- Authenticated request to set Service Definition for the name
- Gated by Service Definition Authorization Policy
- Specific Execution Policy for valid service containers

```
package policy
default allow := false

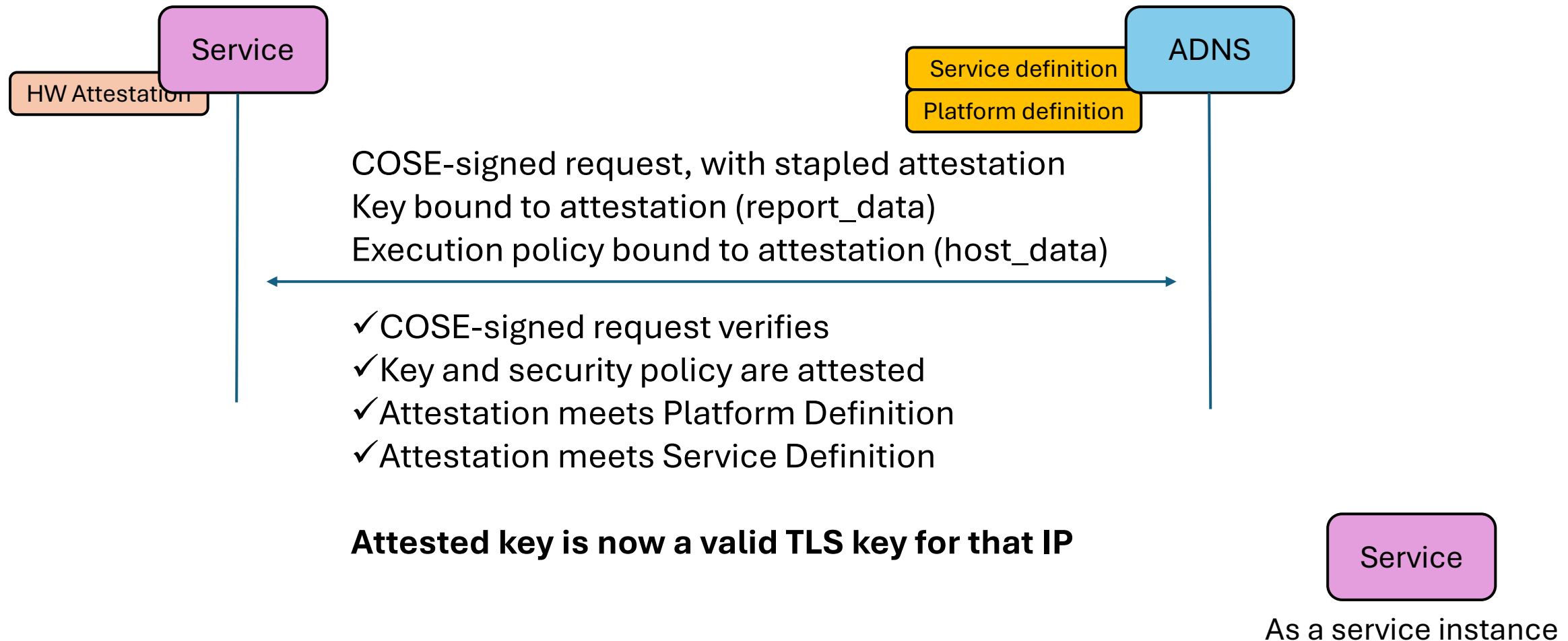
host_data_valid if {
    input.attestation.host_data == "4f4448c67f3c8dfc8de8a5e37125d807dadcc41f06cf23f615dbd52eec777d10"
}

allow if {
    host_data_valid
}
```



As the service owner

Service registers with ADNS



Demo: Service registers with ADNS



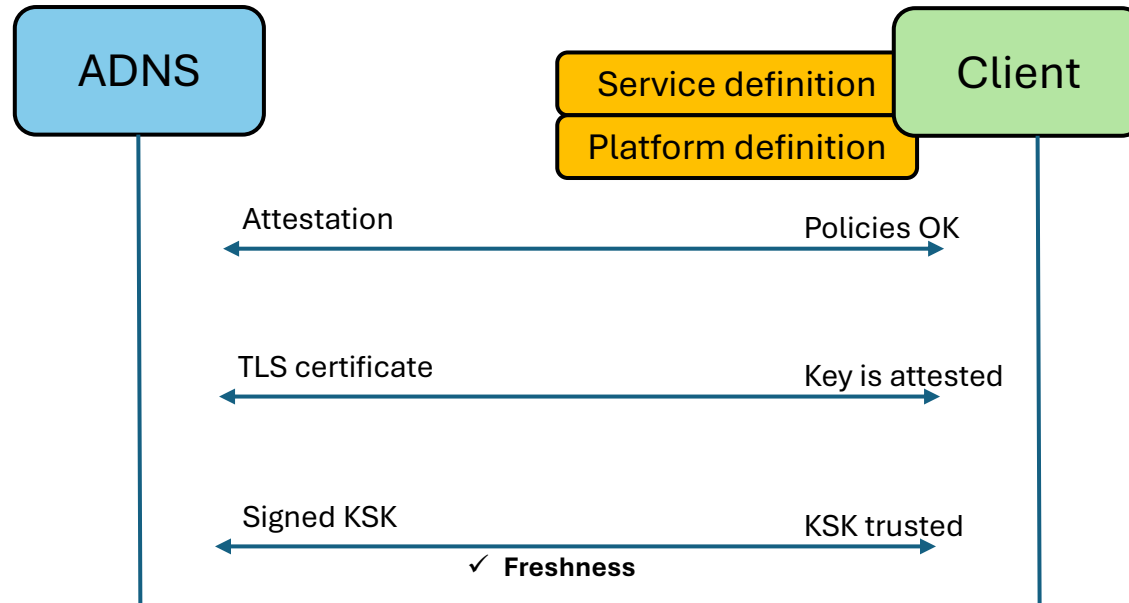
A terminal window with a pink title bar labeled "Service (10.0.1.5)". The terminal text shows the user is root@SandboxHost-638950948511381374 in the directory ~/workspace/ccfdns/demo/server, and they have just entered the command export ADNS_URL=10.0.1.4:1443.

```
root@SandboxHost-638950948511381374 [ ~/workspace/ccfdns/demo/server ]# export ADNS_URL=10.0.1.4:1443
```

Service

As a service instance

Bootstrap trust in ADNS



```
# 1. Check server is running on a valid platform

attestation, endorsements, uvm_endorsements = convert_inputs(
    *fetch_adns_attestation(args.adns)
)
product_name, report, did, feed, svn = verify_snp_attestation(
    attestation, endorsements, uvm_endorsements
)
platform_policy_input = {
    "attestation": {
        "product_name": product_name,
        "reported_tcb": {
            "hexstring": pack_tcb(report.reported_tcb).hex(),
        },
    },
    "uvm_endorsements": {
        "did": did["id"],
        "feed": feed,
        "svn": svn,
    },
}
check_policy(PLATFORM_POLICY, platform_policy_input)

# 2. Check server is running a valid service code

service_policy_input = {
    "attestation": {
        "host_data": report.host_data.hex(),
    },
}
check_policy(SERVICE_POLICY, service_policy_input)

# 3. Fetch signed KSK from server and verify it against the attestation

receipt, tls_key_digest = fetch_adns_ksk_receipt(args.adns)
attested_node_key_digest = report.report_data[0:32]
assert tls_key_digest == attested_node_key_digest

verify_receipt(receipt, attested_node_key_digest.hex())

# 4. DNS-resolved KSK matches the attested KSK

ksk_dns = poll_ksk_from_adns(args.adns, "acidns10.attested.name.")
ksk_digest = receipt["leaf_components"]["claims_digest"]

assert ksk_dns == ksk_digest, f"KSK mismatch: {ksk_dns} != {ksk_digest}"

print("Verified zone KSK is created and owned by a valid ADNS instance")
```

Demo: bootstrap trust in ADNS



```
Client
root@SandboxHost-638951069770812922 [ ~/workspace/ccfdns/demo/client ]# |
```

Client

As a client

Delegate attested zone to ADNS

- Use BIND9 with pinned KSK

```
KSK=$(cat ksk.pinned)

// Forward zone for .acidns10.attested.name domain
zone "acidns10.attested.name" {
    type forward;
    forward only;
    forwarders {
        $ADNS_URL port 5353;
    };
};

trust-anchors {
    "acidns10.attested.name" initial-key 257 3 14 "$KSK";
};

options {
    dnssec-validation yes;
};
```

Client

As a client

Demo: delegate attested zone to ADNS



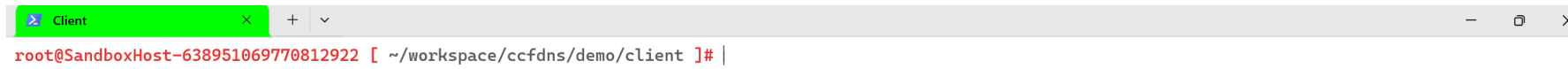
A terminal window with a dark background. The title bar is green and contains the text 'Client: BIND'. The terminal shows a shell prompt '(reverse-i-search)`exp`:' followed by the command 'export ADNS_URL=10.0.1.4'.

```
(reverse-i-search)`exp`: export ADNS_URL=10.0.1.4
```

Client

As a client

Demo: Discover the service IP and TLS key



A terminal window with a green title bar labeled 'Client'. The terminal shows a red prompt 'root@SandboxHost-638951069770812922' followed by a red bracket containing the path '~/.workspace/ccfdns/demo/client' and a red prompt character '#'. A mouse cursor is positioned at the end of the line.

```
root@SandboxHost-638951069770812922 [ ~/.workspace/ccfdns/demo/client ]# |
```

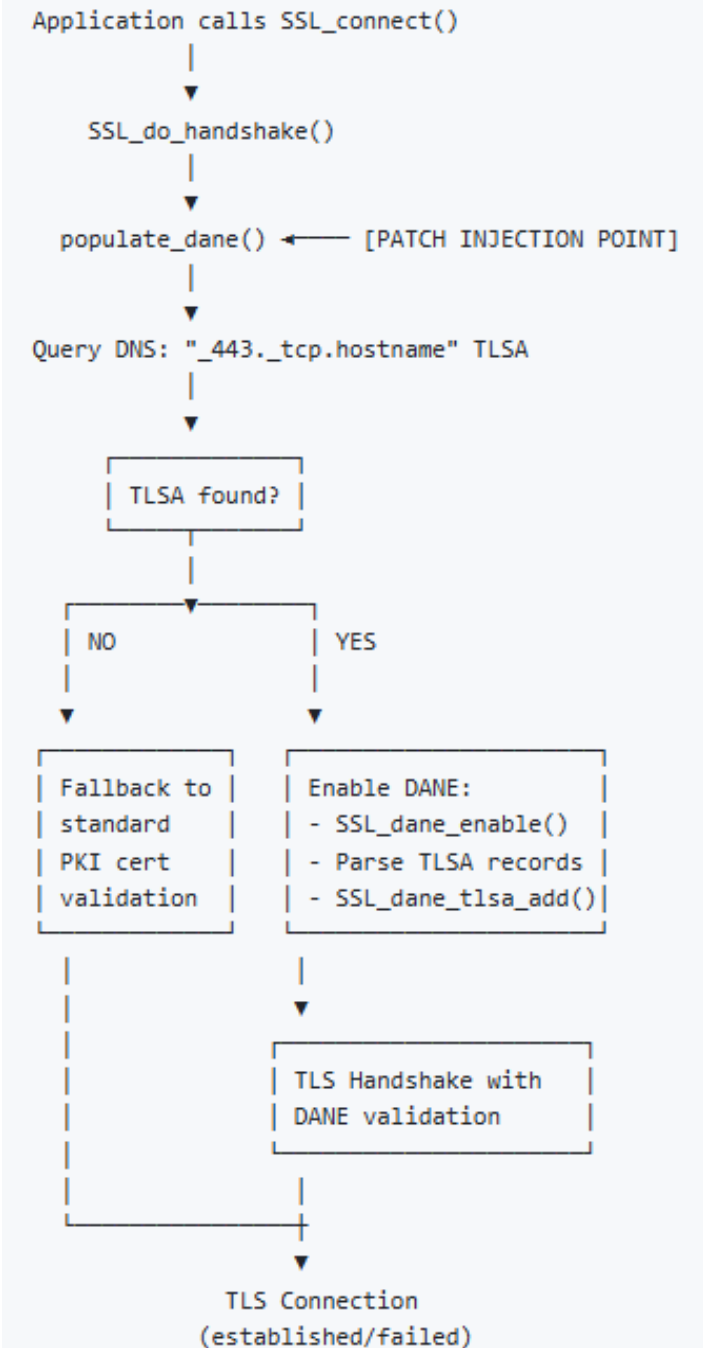
Client

As a client

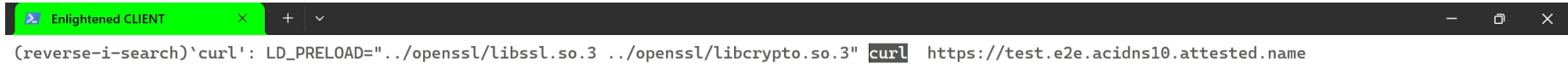
Establish TLS session to service using resolved key

- No typical HTTPS client uses DNSSEC + TLSA
- But they often use ([patched?](#)) OpenSSL

```
static void enable_dane(SSL *ssl) {  
    hostname = SSL_get_servername(ssl, TLSEXT_NAMETYPE_host_name);  
    tlsa_records = resolve_again_with_ask_tlsa_records_(hostname)  
    for (entry : tlsa_records) {  
        add_key(...);  
    }  
  
    enable_dane(ssl);  
}  
  
static int ssl_do_handshake_intern(void *vargs) {  
    // ...  
    enable_dane(...);  
    // ...  
}
```



Demo: Establish TLS session using resolved key



A terminal window titled "Enlightened CLIENT" with a green title bar. The window contains a single line of text: `(reverse-i-search)`curl': LD_PRELOAD=" ../openssl/libssl.so.3 ../openssl/libcrypto.so.3" curl https://test.e2e.acidns10.attested.name`. The terminal has a dark background and standard window controls (minimize, maximize, close) on the right.

Client

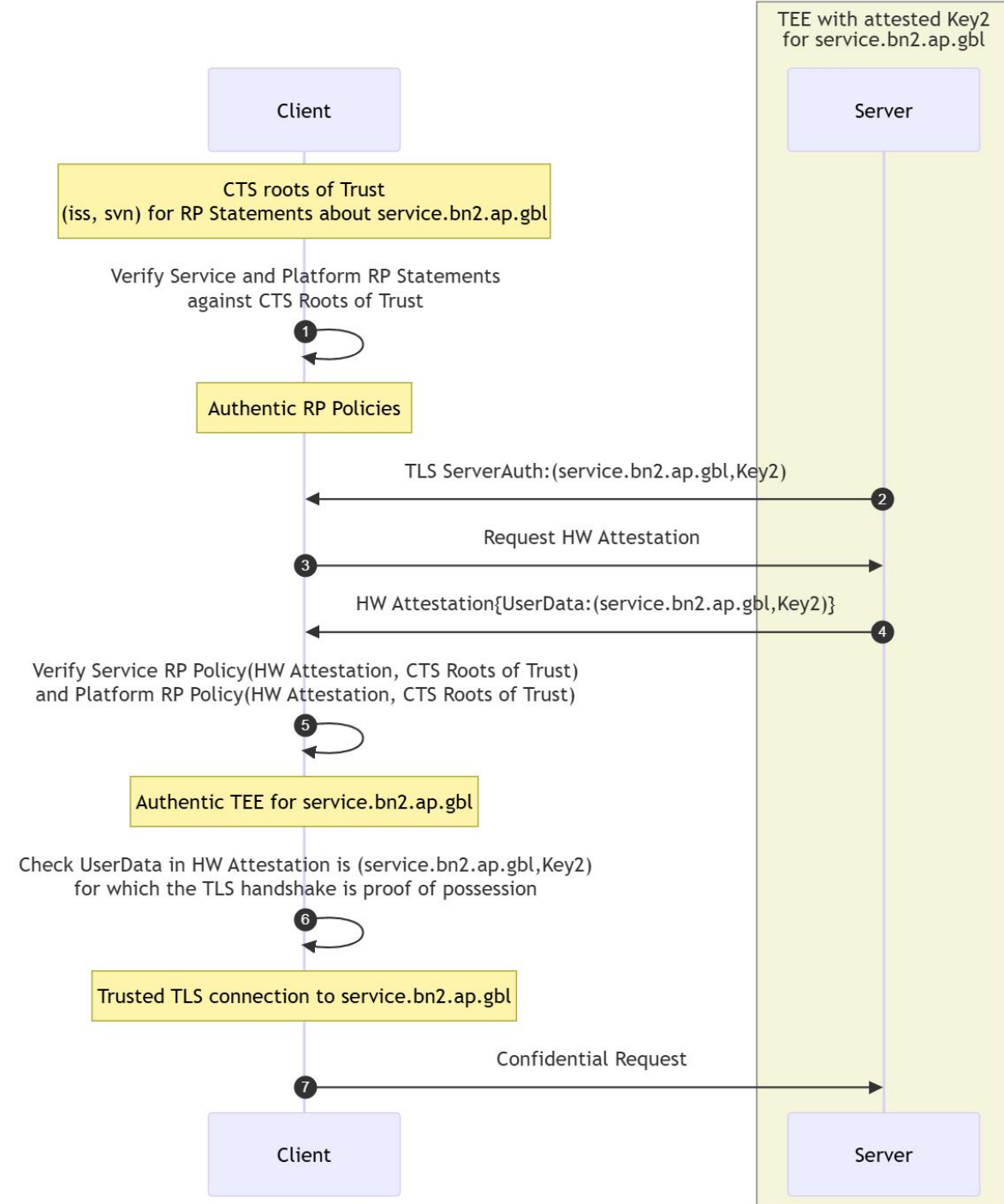
As a client

References

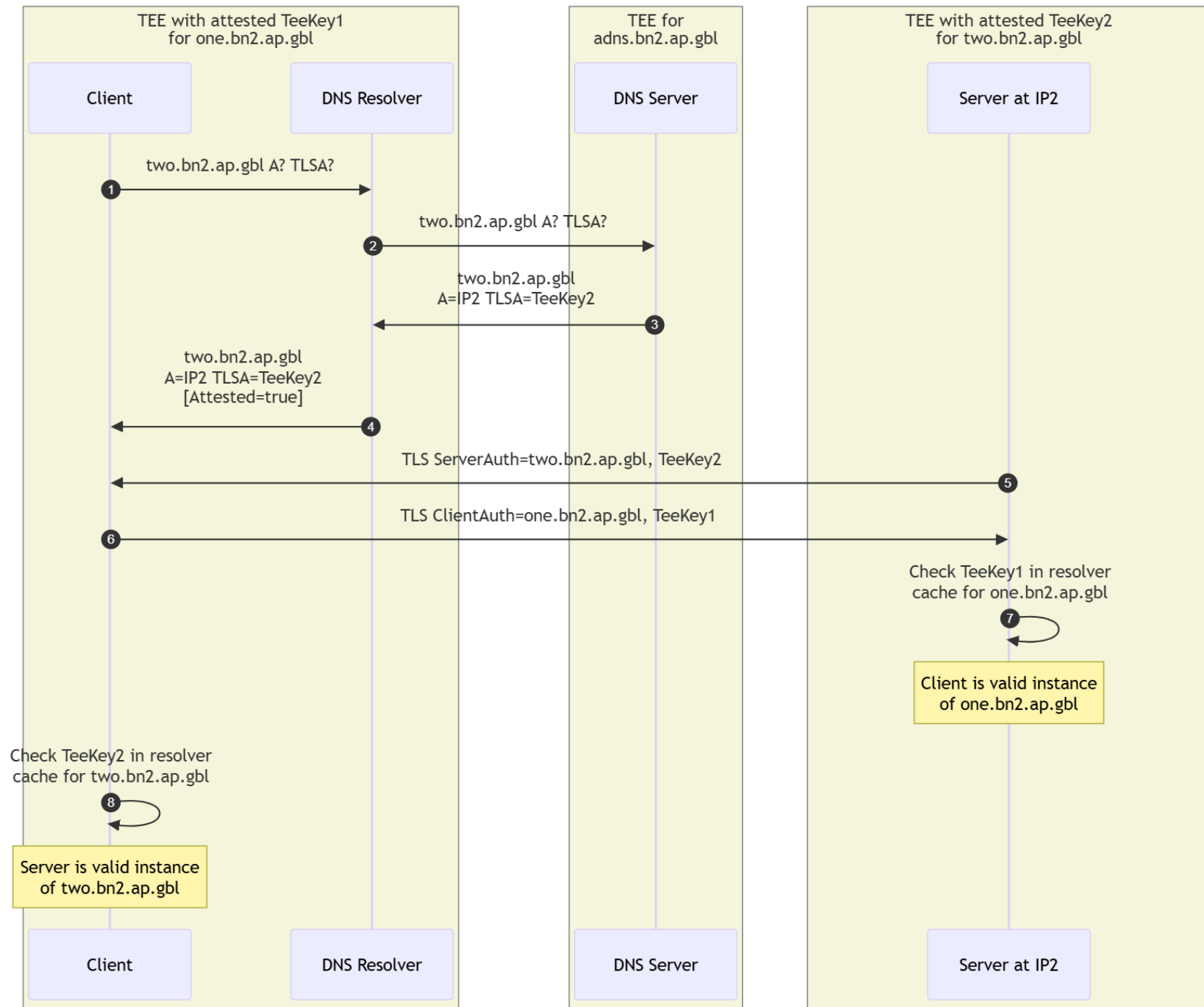
- Attested DNS code: <https://github.com/microsoft/ccfdns>
- Transparent Attested DNS for Confidential Computing Services ([USENIX 2025](#))
- DNSSEC: [RFC9364](#), [RFC4033](#), [4034](#), [4035](#) and others
- DANE: [RFC6698](#), [7218](#), [7671](#) and [8749](#)
- CCF: [ccf.dev](#)

Questions?

Gaining Trust from Scratch in a Confidential Service (baseline)



As a Service, How do I Communicate with Other Services?



How to I register TEEs for my Service?

